

Assignment 3 — Bridge Pattern

Astana IT University | School of Software Engineering | ShP-2216 Software Design Patterns

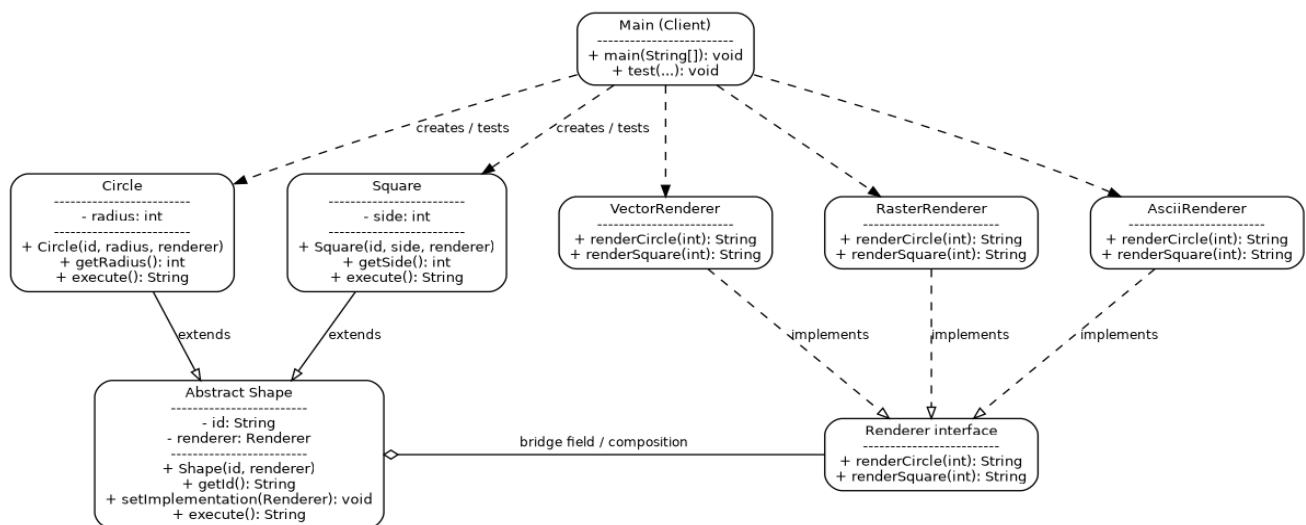
Topic A — Drawing | Student: Muratbek Ibrai | Group: SE-2529

1. Design and Bridge rationale

This application separates two independently varying dimensions. The abstraction hierarchy is Shape → Circle / Square; the implementation hierarchy is Renderer → VectorRenderer / RasterRenderer / AsciiRenderer. The abstract base stores a Renderer reference supplied through its constructor. Circle and Square keep their own domain data and delegate rendering through that interface reference.

Bridge is appropriate because either hierarchy can grow without creating a separate class for every combination. A new shape can reuse the existing renderers, while a new renderer can support both existing shapes. The same Shape object can also switch its implementation at runtime through `setImplementation(Renderer)`.

2. UML class diagram



The composition link on Shape → Renderer represents the interface-typed bridge field. The three renderer classes realize Renderer. The diagram matches the submitted source structure.

3. Required demonstration

Check	Case	Expected result
T1	Circle + Vector	VECTOR circle radius=2
T2	Circle + Raster	RASTER circle radius=2
T3	Square + Vector	VECTOR square side=3
T4	Square + Raster	RASTER square side=3
T5	Same Circle: Vector → Raster	sameObject=true; state unchanged; result changes
T6	Circle + ASCII	ASCII circle radius=2
T7	Square + ASCII	ASCII square side=3

T5: Main stores the original Circle reference, executes with VectorRenderer, replaces the implementation with RasterRenderer, then uses `originalReference == afterReference`. It also compares the ID and radius before/after. The submitted output reports `sameObject=true` and `stateUnchanged=true`.

4. Five annotated Clean Code excerpts

1. Separation of responsibilities. Renderer defines only low-level rendering operations. Circle and Square own shape state and delegation. This prevents rendering details from being embedded in the abstraction. [Renderer.java, Circle.java]
2. Meaningful names. Names such as setImplementation, renderCircle, renderSquare, radius, and side communicate both the Bridge roles and the domain model. [Shape.java, Renderer.java]
3. Focused methods. Each execute() method performs one delegation, while each renderer method returns one rendering description. Small methods make the design easy to trace during defense. [Circle.java, Square.java, renderer classes]
4. No duplicated workflow. Main.test(...) centralizes expected-versus-actual comparison and PASS/FAIL calculation, so T1-T4 and T6-T7 do not duplicate test-reporting logic. [Main.java]
5. Encapsulated state. Shape ID, renderer, radius, and side are private. The renderer reference changes only through setImplementation, while domain values remain unchanged during T5. [Shape.java, Circle.java, Square.java]

5. Extension with I3

The base commit is the working I1/I2 solution. The extension adds AsciiRenderer and changes Main only to demonstrate T6 and T7 and update the summary. Existing abstraction classes, the Renderer interface, VectorRenderer, and RasterRenderer remain unchanged. The source diff was generated from the base commit to the submitted source commit.

Base commit: **1e8ab6baf9eef9ac5b888e9c6e2f026fe112364a9**

Submitted source commit: **eb8bd5902fd6765e8c9d4108e5115fd202d67d14**

6. Bridge compared with Adapter + one trade-off

Bridge separates an abstraction from its implementation so both can vary independently. In this application, Shape and Renderer are intentionally designed as the two sides of that separation. Adapter instead makes an existing class or interface usable through another expected interface when the contracts do not match. Here, there is no incompatible legacy API to convert; Renderer is deliberately the implementation contract of the Bridge.

Trade-off: Bridge introduces an extra interface and indirection, so a very small one-implementation problem can become more verbose. The benefit is lower coupling and independent extension of both dimensions.

7. Sources

- [1] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. Bridge and Adapter patterns.
- [2] Astana IT University, School of Software Engineering. Assignment 3 | Bridge Pattern, 2026–2027, pp. 1–6. Requirements, checks, rubric, and submission format.
- [3] OpenAI ChatGPT. Used as a reference/clarification aid during preparation; the submitted code is intended to be understood and defended by the student.

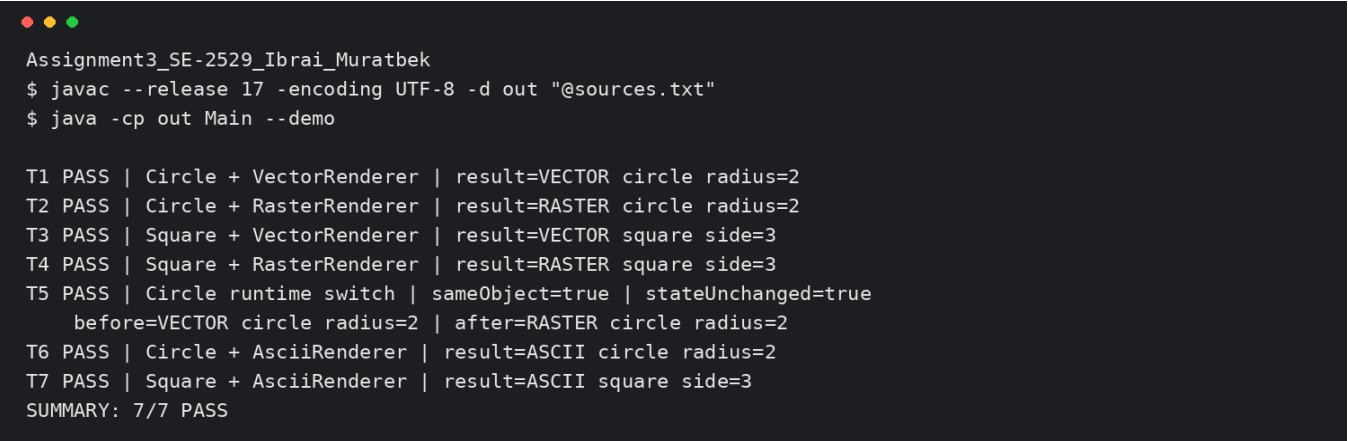
8. Delivery, screenshots, and user experience

The submission is designed for quick evaluator use. It is delivered as a ZIP for Moodle, with all required files directly at the ZIP root. The Java demo is non-interactive and needs no GUI, network, database, or external dependency. The evaluator can extract the ZIP, read README.md, run two commands, and immediately see T1-T7 and the calculated summary.

Run commands

```
javac --release 17 -encoding UTF-8 -d out "@sources.txt"
java -cp out Main --demo
```

Successful run — screenshot generated from the actual checked console output



```
Assignment3_SE-2529_Ibrai_Muratbek
$ javac --release 17 -encoding UTF-8 -d out "@sources.txt"
$ java -cp out Main --demo

T1 PASS | Circle + VectorRenderer | result=VECTOR circle radius=2
T2 PASS | Circle + RasterRenderer | result=RASTER circle radius=2
T3 PASS | Square + VectorRenderer | result=VECTOR square side=3
T4 PASS | Square + RasterRenderer | result=RASTER square side=3
T5 PASS | Circle runtime switch | sameObject=true | stateUnchanged=true
      before=VECTOR circle radius=2 | after=RASTER circle radius=2
T6 PASS | Circle + AsciiRenderer | result=ASCII circle radius=2
T7 PASS | Square + AsciiRenderer | result=ASCII square side=3
SUMMARY: 7/7 PASS
```

Submission structure example

Submission structure

```
Assignment3_SE-2529_Ibrai_Muratbek/
├── src/
│   ├── Main.java
│   ├── Shape.java
│   ├── Circle.java
│   ├── Square.java
│   ├── Renderer.java
│   ├── VectorRenderer.java
│   ├── RasterRenderer.java
│   └── AsciiRenderer.java
├── sources.txt
├── README.md
├── demo-output.txt
├── extension.diff
├── report.pdf
├── uml-diagram.png
└── screenshots/
    ├── demo-output.png
    └── project-structure.png
```

Error / unsupported behavior. An unsupported command prints a short usage line. Missing JDK tools or running outside the project root prevents compilation and is documented in README.md. Real GUI rendering, networking, databases, and hardware are intentionally unsupported because the assignment asks for local string simulation. This keeps the user experience reproducible and simple.